Пакет Image Processing Toolbox и Гистограммы

1. Пакет Image Processing Toolbox

"Пакет Image Processing Toolbox – это набор функций, которые расширяют возможности числовой вычислительной среды MATLAB. Пакет поддерживает широкий спектр операций по обработке изображений".

- ✓ Пакет Image Processing Toolbox обеспечивает набор инструментов, которые позволяют визуализировать и работать с изображениями;
- ✓ Некоторые классические вещи, которые позволяет делать пакет Image Processing Toolbox:
  - гистограммную эквализацию (`imhist`);
  - фильтрацию (`imfilter`);
  - быстрое преобразование Фурье (`fft2`);
  - преобразование цветного изображения в полутоновое (`rgb2gray`);
  - обнаружение краев (`edge`);
- ✓ Для просмотра демо-версии того, что можно делать с помощью пакета Image Processing Toolbox, необходимо воспользоваться командой `iptdemos`.

1.1. Чтение и визуализация изображений

- ✓ Существует ряд фотографий, которые поставляются в составе инструментария для обработки изображений (IPT). Чтобы получить список изображений следует набрать:

```
help imdemos
```

или

```
doc imdemos
```

- ✓ Если требуется просмотреть любую из этих фотографий, можно использовать функцию `imshow`, которая открывает ее в отдельном окне просмотра. Например:

```
imshow('football.jpg');
imshow('coins.png');
imshow('autumn.tif');
imshow('board.tif');
```

- ✓ Если требуется изменить или улучшить одну из этих фотографий, то необходимо сохранить ее во временном массиве. Это можно сделать, используя `imread`. Например, чтобы создать массив `I`, который содержит информацию об изображении под названием `pout.tif`, используется следующий синтаксис:

```
I=imread('pout.tif');
```

чтобы потом просмотреть изображение в `I`:

```
imshow(I);
```

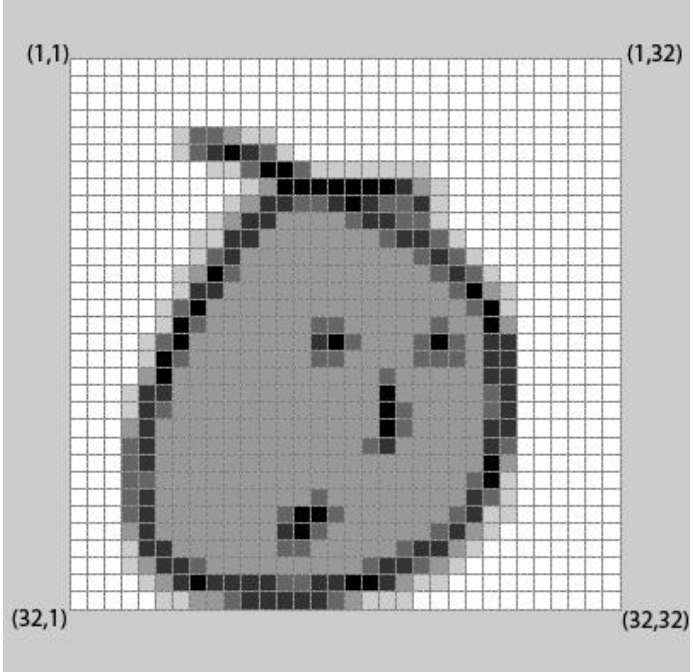
- ✓ Следующая таблица предназначена для обобщения команд, которые используются для чтения и отображения изображений

Команда	Описание
`I=imread('filename');`	Чтение изображения из `'filename'` и сохранение его в `I`
`[I,map]=imread('filename.gif');`	Чтение индексированного изображения и сохранение его в `I` с цветовой картой `map`
`imshow(I);`	Отображение изображения, хранящемуся в `I` в

	отдельном окне
`imshow(I, map);`	Отображение индексированного изображения, хранящемуся в `I` в отдельном окне, используя цветовую карту `map`
`imtool(I);`	Отображает изображение. Позволяет узнать значение пикселя в области и т. д. Отображение индексированных изображений подобно `imshow`

1.2. Что хранится в изображении?

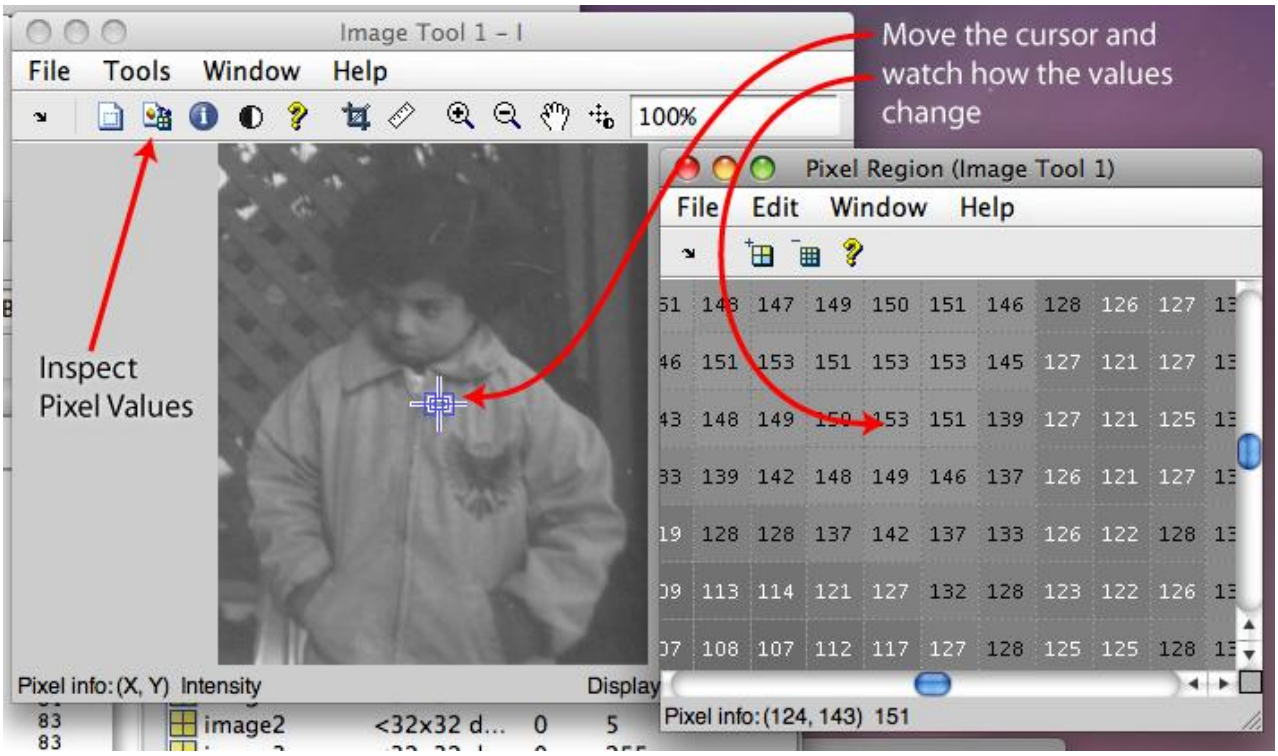
Когда мы оперируем изображениями, мы все еще имеем дело с матрицами, которые могут быть доступны через индексы. Верхний левый угол изображения индексируется как (1, 1). Общий формат индексирования (*строка*, *столбец*) или (y, x). Будьте осторожны, потому что вы привыкли работать в декартовых (x, y) координатах. Следующая диаграмма показывает индексы всех четырех углов небольшой картинки:



Все основные операции с матрицами, которые вы узнали с прошлых занятий, до сих пор применяются. Что вы должны спросить у самого себя – что же все-таки хранится в матрице изображения? Будем использовать полутоновое изображение (в градациях серого) (`"pout.tif"`) в качестве примера. Это изображение хранится в матрице размерностью 291 × 240. Это означает, что 291 строк и 240 столбцов. Каждый элемент соответствует одному значению пикселя.

В случае `"pout.tif"`, значения от 0 до 255. Нуль – это полная чернота и 255 – это белый цвет. Значения от нуля до 255 являются оттенками серого.

- ✓ Предположим, что изображение `"pout.tif"` было считано в матрицу `I`.
- ✓ Если набрать `imtool(I)`, вы начнете работать с инструментом изображения.
- ✓ Вы можете получить для просмотра крупным планом пиксели той или иной области, используя опцию “Inspect pixel values” из панели инструментов.
- ✓ Вы можете изменить размер области просмотра, перетащив углы области на основном изображении. Если окно достаточно мало, крупным планом будут отображаться значения пикселей.
- ✓ Нажмите по середине окна и перемещайте курсор по изображению, чтобы увидеть, что более яркие области ближе к 255, а темные участки ближе к 0.



Примечание: строка “Pixel info”, которая находится в нижней части, отображает значение в формате (столбец, строка) или (*x*, *y*). Это отличается от синтаксиса индексирования системы MATLAB, который имеет формат (строка, столбец) или (*y*, *x*).

1.3. Отображение информации об изображении

В следующей таблице приведены несколько способов того, как получить информацию об изображении. Они не являются специфическими для Image Processing Toolbox.

Команда	Описание
`whos`	Для получения информации о размере, типе и байтах всех переменных
`whos I`	Для получения информации об изображении, хранящемуся в `I`
`size(I)`	Для получения размера изображения, сохраненном в `I`
`class(I)`	Для получения типа данных, хранящихся в `I`

Цветовые форматы

Существуют различные типы данных, хранящихся в массиве изображения (или матрице). В приведенной ниже таблице представлены различные типы изображений и значений.

Тип изображения	Значения/Диапазон интенсивности
Двоичное/Binary	o логический (булевый) массив 0-й и 1-ц; o 0 – это черное и 1 – это белое.
Индексированные / Indexed (pseudocolor)	o значения в изображении указывают на цветовую карту; o диапазон значений, где p является длиной цветовой карты: – массивы типа single или double [1, p] – logical, uint8, или uint16 [0, p-1]
Полутоновые / Grayscale (intensity)	o значения имеют следующий диапазон: – массивы типа single или double [0, 1]; – uint8 [0, 255]; – uint16 [0, 65 535]; – int16 [-32 768, 32 767];
Цветные/Truecolor	o значения имеют следующий диапазон:

(RGB)	– массивы типа single или double [0, 1]; – uint8 [0, 255]; – uint16 [0, 65 535].

Цветные/Truecolor

До сих пор мы имели дело с черно-белыми изображениями и как известно существуют различные типы изображений. Большинство из них работают аналогично. Для каждого пикселя есть одно значение из диапазона значений соответствующих типов данных, хранящихся в двумерной матрице. Для представления цвета следует добавить еще одно измерение. Например, если считывается изображение `"autumn.tif"` в `I2`, то для просмотра информации в `I2`, используется:

```
I2=imread('autumn.tif');
whos I2
```

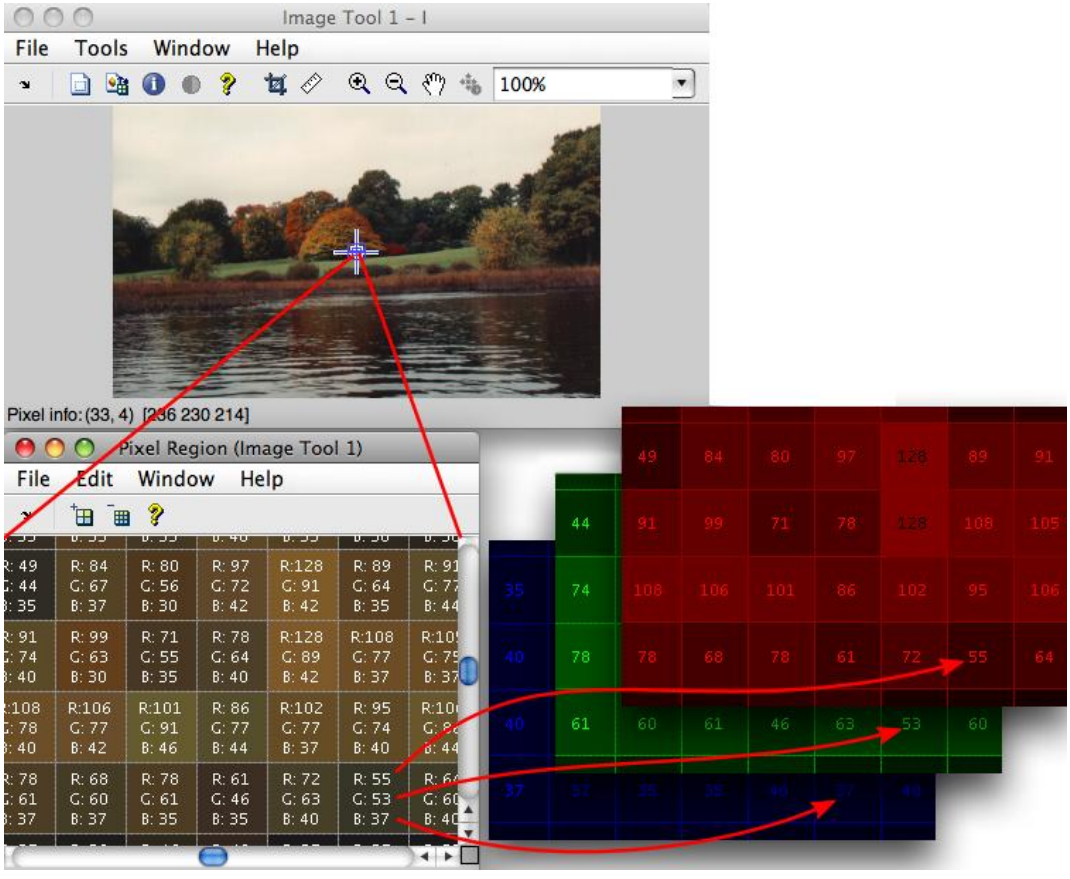
The image size is: `206 × 345 × 3`

Дополнительное измерение это цветовой компонент. Для каждого пикселя, мы теперь имеем представление об интенсивности красного, зеленого и синего цветов. В `autumn.tif`, 255 представляет наибольшую интенсивность, а 0 означает самую низкую интенсивность. Например, чтобы сделать чисто красным, красный будет 255, зеленый будет 0, а синий будет 0.

В пикселе (93, 180) файла `autumn.tif`,

- ✓ интенсивность красного представлена как I2(93,180,1);
- ✓ интенсивность зеленого представлена как I2(93,180,2);
- ✓ интенсивность синего представлена как I2(93,180,3).

Следующая диаграмма представляет три цветовые компоненты в пикселях. Часто они представлены в виде каналов, как показано на рисунке ниже. Как правило, есть красный, зеленый и синий каналы.



Если бы вы хотели только просмотреть в красном канале, то какие из следующих утверждений вы используете?

```
a.  imshow( I2(1, :, :) );
b.  imshow( I2(:, :, 1) );
c.  imshow( I2(:, 1, :) );
```

Как выглядит изображение?

Что произойдет, если попробовать другой цветовой канал?

1.4. Конвертирование между различными форматами

В следующей таблице приведены команды, предоставляемые пакетом Image Processing Toolbox, которые позволяют преобразовывать между различными форматами, приведенных выше.

Преобразование форматов изображений (в скобках указывается имя файла с изображением, который следует преобразовать)	
MATLAB команда	**Операция**
`gray2ind()`	Строит индексированное изображение из полутонового.
`ind2gray()`	Строит полутоновое изображение из индексированного.
`ind2rgb()`	Строит RGB изображение из индексированного.
`mat2gray()`	Создает из матрицы чисел полутоновое изображение, осуществляя необходимые операции масштабирования для приведения к виду, принятому для полутоновых изображений.
`rgb2gray()`	Строит полутоновое изображение из RGB.
`rgb2ind()`	Строит индексированное изображение из RGB.
`im2bw()`	Преобразование полутонового/индексированного/RGB изображения в черно-белое бинарное.
`dither()`	Преобразование полутонового/индексированного/RGB изображения в индексированное методом дрожания с произвольной цветовой картой. По умолчанию преобразование в черно-белое изображение, если не указана цветовая карта.

Команда `mat2gray` полезна, если у вас есть матрица, представляющая изображение, но значения, представляющие оттенки серого находятся в диапазоне, скажем, между 0 и 1000. Команда `mat2gray` автоматически перемасштабирует все записи так, что они попадут между 0 и 255 (если вы используете uint8 класс) или 0 и 1 (если вы используете double класс).

Вот несколько примеров того, как использовать некоторые преобразования команд:

```
%Начните с цветного изображения
img=imread('onion.png');
imshow(img);
```



```
%Преобразуйте в двоичное/черно-белое
img_bin=im2bw(img);
imshow(img_bin);
```



```
%Преобразуйте в двоичное/черно-белое
%с произвольным порогом (по умолчанию 0.5)
img_bin2=im2bw(img, 0.3);
imshow(img_bin2);
```



```
%Преобразуйте в полутоновое
img_gray=rgb2gray(img);
imshow(img_gray);
```



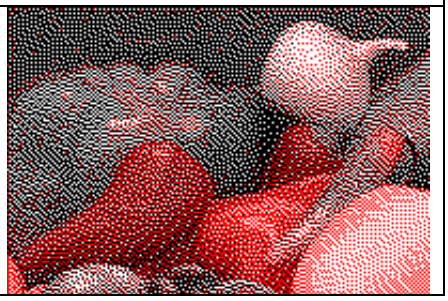
```
%Приведите к индексированному цвету
[img_indexed, map]=rgb2ind(img,8);
imshow(img_indexed, map);
```



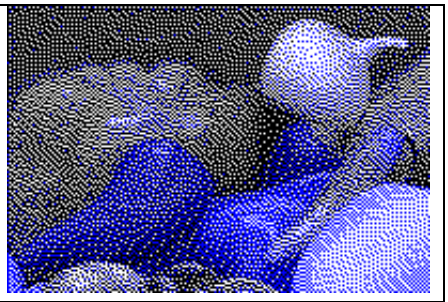
```
%Приведите к индексированному цвету
%без сглаживания
[img_nodither, map2]=rgb2ind(img,8, 'nodither');
imshow(img_nodither, map2);
```



```
%Приведите к индексированному цвету,
%используя произвольную цветовую карту
map_custom=[
   1 1 1;  %white
   1 0 0;  %red
   0 0 0]; %black
img_custommap=dither(img, map_custom);
imshow(img_custommap, map_custom);
```



```
%Конечно же вы можете менять цветовые карты...
map_custom2=[
   1 1 1;  %white
   0 0 1;  %blue
   0 0 0]; %black
imshow(img_custommap, map_custom2);
```



1.5. Сохранение изображений

Для сохранения изображения, используйте `imwrite` функцию. Параметры, которые передаются функции, зависят от формата файла и типа изображения.

Сохранение двоичных, полутоновых или цветных изображений

При сохранении двоичного, полутонового или цветного изображения все, что требуется указать, так это имя матрица и имя файла. Например:

```
imwrite(img_bin, 'onion_bw.png')
```

В этом примере изображение будет сохранено как 1-битовое изображение в png-формате. Формат автоматически определяется из расширения файла.

Сохранение индексированного изображения

Когда сохраняется индексированное изображение очень важно обеспечить цветовую карту подобно следующему:

```
imwrite(img_indexed, map, 'onion.gif');
```

Примечания

- ✓ Если вы сохраняете индексированные изображения в формат, который не поддерживает индексированные изображения, то они будут преобразованы в совместимый формат автоматически. С JPEG типами файлов JPEG это может привести к нежелательному результату.
- ✓ Если вы забыли указать цветовую карту, изображение все равно будет сохранено – индексированные номера будут интерпретироваться как оттенки серого и может выглядеть хаотично. Если ваше изображение имело маленькую цветовую карту, , результат может выглядеть черным.

Опции

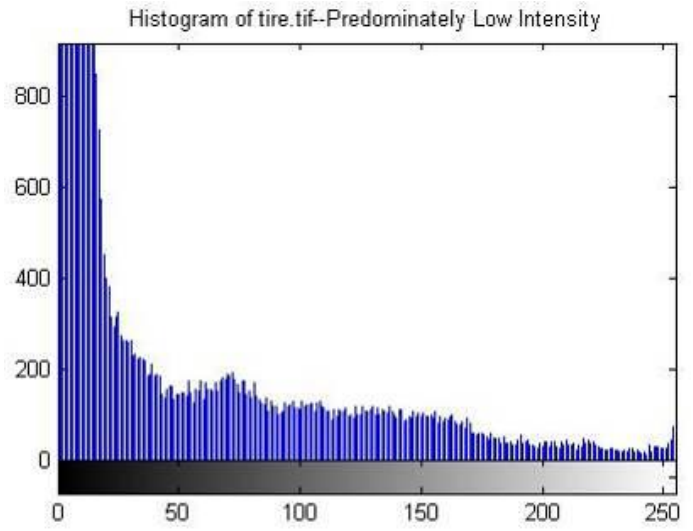
Существуют различные опции, которые могут быть отправлены функции `imwrite` для управления того, как различные типы файлов будут сохранены. Например, вы можете изменить уровень сжатия JPEG следующим образом:

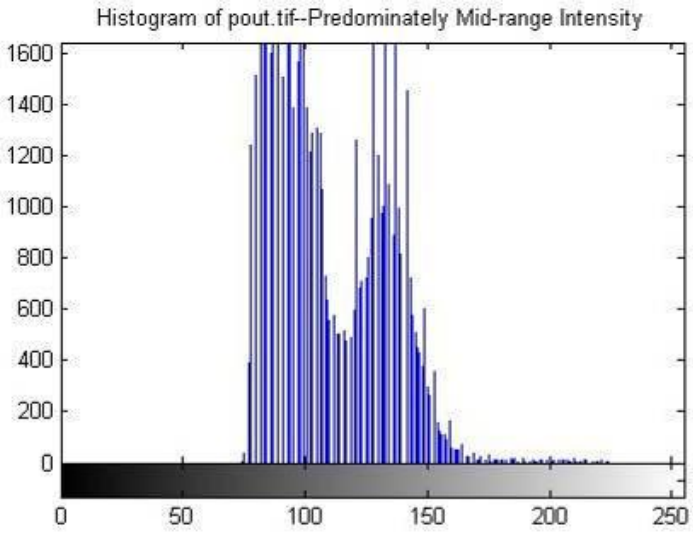
```
imwrite(img, 'onion.jpg', 'Quality', 25);
```

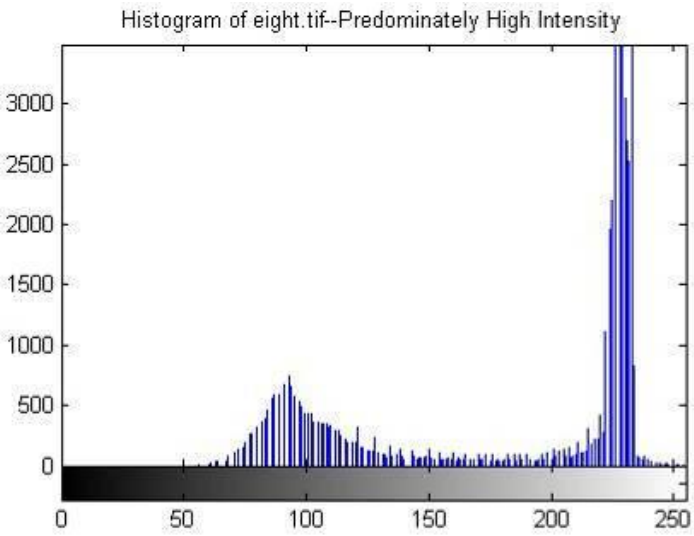
2. Гистограммы

Гистограммы – способ визуализации преобладающей интенсивности изображения. Гистограмма распределения интенсивностей пикселей является одной из наиболее важных характеристик изображения. Именно гистограмма, в некоторых случаях, может служить основой для выбора того или иного метода обработки изображения.

Какое изображение следует ожидать темным или светлым?







Обратите внимание, что ось *x* указывает на значения интенсивности от 0 до 255 (эти изображения являются типа uint8) .Ось *y* изменяется в зависимости от количества пикселей в изображении, и как распределение их интенсивности.

MATLAB легко выводит гистограммы изображений с помощью функции `imhist(I)`. Приведенные выше графики были созданы с помощью следующих команд:

```
tire=imread('tire.tif');
```

```
figure,imhist(tire);
title('Histogram of tire.tif--Predominately Low Intensity');
pout=imread('pout.tif');
figure,imhist(pout);
title('Histogram of pout.tif--Predominately Mid-range Intensity');
eight=imread('eight.tif');
figure, imhist(eight);
title('Histogram of eight.tif--Predominately High Intensity');
```

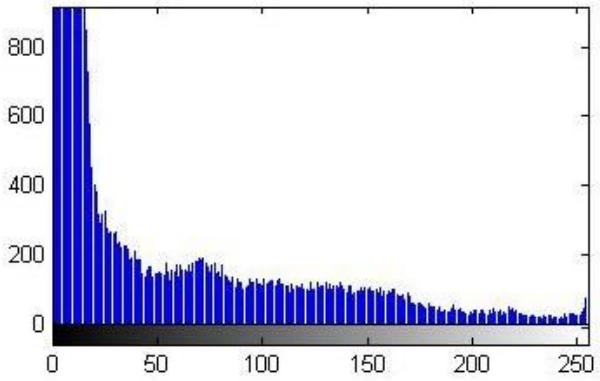
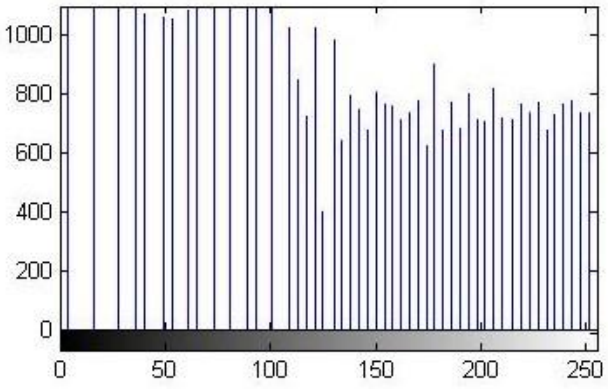
2.1. Гистограммная эквализация

Идея гистограммной эквализации состоит в том, что мы пытаемся равномерно распределить частоты интенсивности пикселов так, что весь диапазон интенсивности используется наиболее полно. Мы стараемся дать каждому пикселю интенсивности равные возможности, таким образом, ***эквализируем*** (***выравниваем***). Специально для изображений с широким спектром значений со средней группировкой вокруг нескольких интенсивностей, гистограммы позволяют улучшить контрастность изображения.

В MATLAB функцией для гистограммной эквализации является `histeq(I)`.

Ниже приведены некоторые примеры гистограммной эквализации, и ее результаты на изображениях, которые имеют преимущественно низкий и средний диапазон интенсивностей:

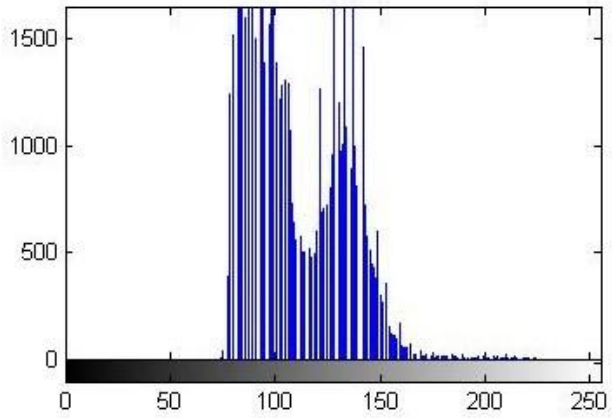
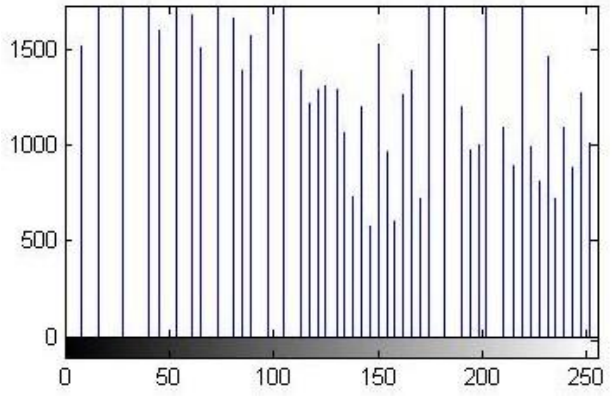
2.1.1. Низкий диапазон интенсивностей

	Изображение	**Гистограмма**
Исходное		
С гистограммной эквализацией		

Приведенные выше диаграммы были созданы с использованием следующих команд:

```
tire=imread('tire.tif');
imshow(tire);
figure, imhist(tire);
tire_eq=histeq(tire);
figure, imshow(tire_eq);
figure, imhist(tire_eq);
```

2.1.2. Средний диапазон интенсивностей

	Изображение	Гистограмма
Исходное		
С гистограммной эквализацией		

Приведенные выше диаграммы были созданы с использованием следующих команд:

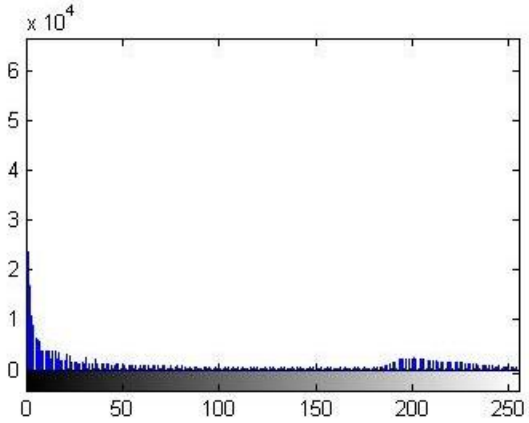
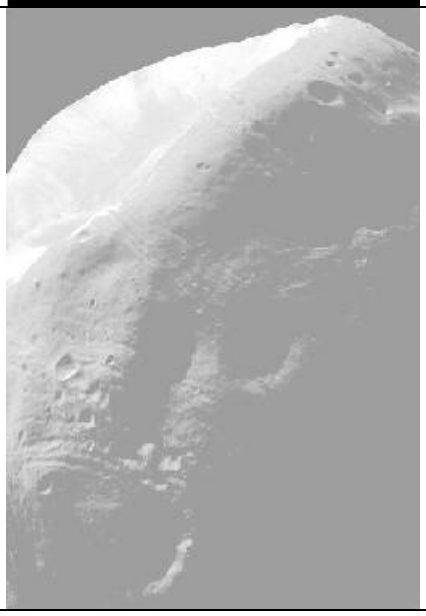
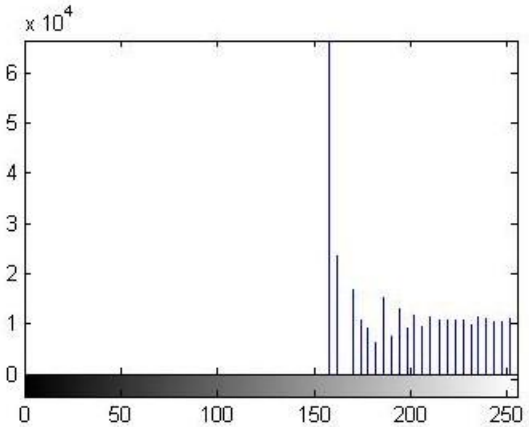
```
pout=imread('pout.tif');
imshow(pout);
figure, imhist(pout)
pout_eq=histeq(pout);
figure, imshow(pout_eq);
figure, imhist(pout_eq);
```

Примечание: `imshow(image,[low, high])` также выполняет растяжение гистограммы: все значения меньшие или равные `low` будут показаны, как черные, все значения выше или равные `high` будут показаны, как белые, остальные будут простираться от 0 до 255.

`imshow(pout, [75, 160])`	

2.2. Гистограммная подгонка

Гистограммная эквализация совершается преобразованием, которое является адаптивным в том смысле, что оно зависит от гистограммы исходного изображения. Однако если функция преобразования уже вычислена, то она не будет меняться, пока не изменится само изображение. Гистограммная эквализация улучшает изображение путем расширения диапазона его уровней до более широкой шкалы яркости. Такая процедура не всегда приводит к удовлетворительному результату. Поэтому в конкретных приложениях полезно уметь задавать форму гистограммы, которую желательно иметь для обработанного изображения. Метод построения обработанного изображения с заданной гистограммой называется *гистограммной подгонкой* или *гистограммной спецификацией*. Исходные изображения предоставлены Агентством NASA.

	Изображение	**Гистограмма**
Исходное		
С гистограммной эквализацией		

Приведенные выше изображения были созданы с использованием следующих шагов:

1. Загрузите изображение из файла `'Fig0310(a)(MoonPhobos).tif'`;
2. Используйте следующий код в командном окне “Command Window”.

```
moon = imread('Fig0310(a)(MoonPhobos).tif');
imshow(moon);
figure, imhist(moon);
```

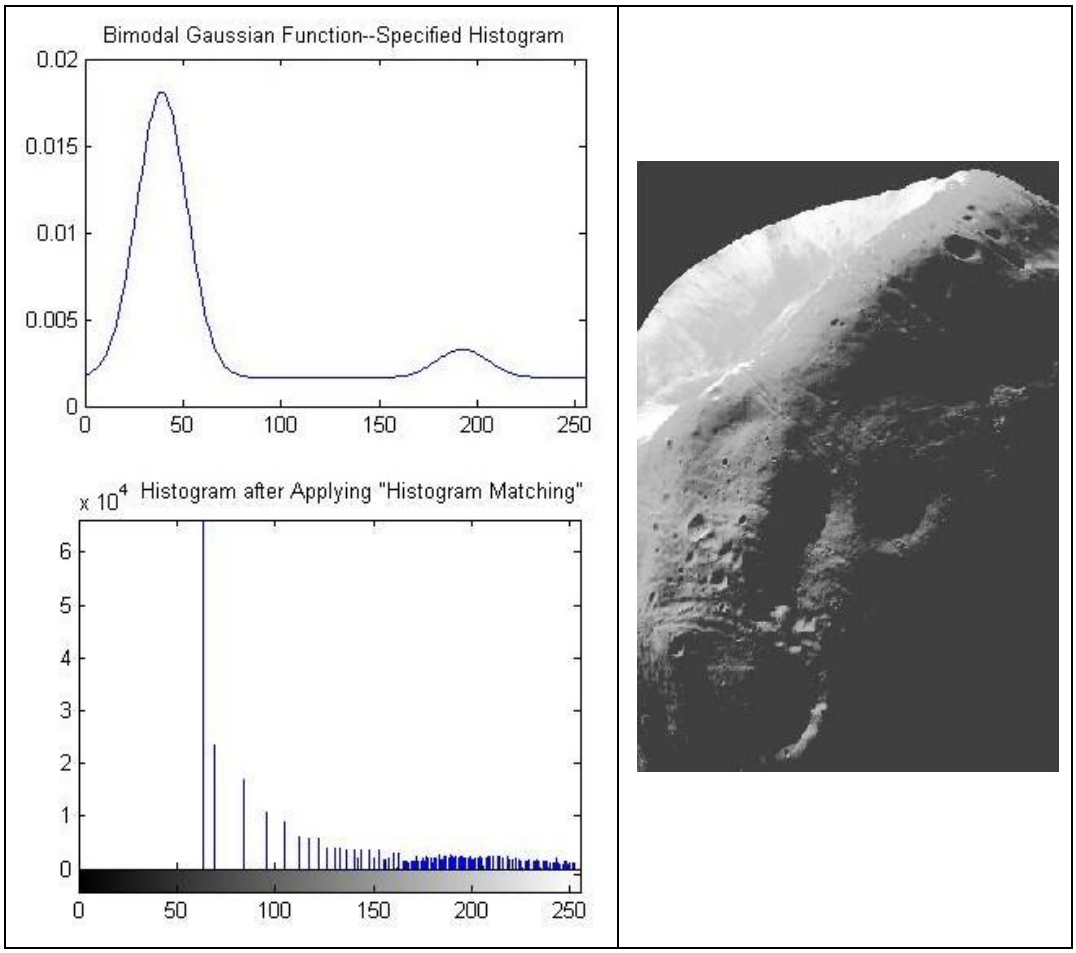
```
moon_eq = histeq(moon);
figure, imshow(moon_eq);
figure, imhist(moon_eq);
```

Вы заметите, что на приведенном выше изображении с гистограммной эквализации присутствует очень мало значений с низкой интенсивностью. Чтобы это исправить, мы будем использовать гистограммную подгонку (или гистограммную спецификацию).

Чтобы обеспечить некоторый фон, гистограммная эквализация пытается создать равную вероятность каждой возникающей интенсивности. Эта равная вероятность представляет собой горизонтальную линию. Если вы хотите подогнать результаты, приведенные выше, вы можете передать горизонтальную линию в качестве второго аргумента `histeq`. Следующий код может быть использован для получения результатов, аналогичных приведенным выше гистограммной эквализации.

```
p(1:256)=1
g=histeq(moon,p);
figure, imshow(g);
figure, imhist(g);
```

С гистограммной подгонкой, мы предоставляем мультимодальную (многоконечную) Гауссову функцию в качестве аргумента `histeq`. В отличие от горизонтальной линии гистограммной эквализации, функция Гаусса указывает на то, что определенный диапазон интенсивности будет иметь больше вероятность возникновения, чем другие. Интуитивно говоря, путем передачи мультимодальной функции Гаусса в качестве второго аргумента `histeq`, измененная гистограмма изображения будет иметь пики, которые примерно соответствуют пикам гауссовой функции. Обратите внимание, это приблизительная подгонка для примера, приведенного ниже:



Для того чтобы создать приведенные выше результаты, были использованы две M-функции:

- ✓ `manualhist` предоставляет пользовательский интерфейс для ввода различных аргументов и просмотра результирующей гистограммы (создается путем вызова функции `twomodegauss`);
- ✓ `twomodegauss` генерирует бимодальную Гауссову функцию, используя семь аргументов.

С учетом указанных выше двух функций в «текущем каталоге», следующие вызовы были сделаны для достижения результатов в приведенной выше таблице:

```
p = manualhist;
%следующие аргументы были введены при запросе
% 0.15 0.05 0.75 0.05 1 0.1 0.002
% - ввод не забудьте закончить символом x
g = histeq(moon, p);
figure, imshow(g);
figure, imhist(g);
```

3. Возвращение к M-файлам

В конце этого раздела следует обратить внимание на функции, написанные в MATLAB, которые могут быть применены для всех преобразований интенсивности, упомянутых выше. Прежде чем сделать это, ниже приводится некоторая справочная информация о M-файлах и функциях.

3.1. Скрипты

В предыдущей лабораторной работе мы кратко рассмотрели M-файлы. В частности, мы разбирали m-файл скрипт. Вспомним:

- ✓ расширением является `.m`;
- ✓ если мы хотим вызвать наш файл `myScript.m`, для запуска его в MATLAB:
  - – мы должны установить в качестве текущей директории размещение файла `myScript.m`;
  - – мы можем сослаться на него, введя его имя без расширения:

```
myScript
```

- ✓ содержимое скрипта является обычным MATLAB кодом.

Сценарии не принимают входные аргументы и не возвращают ничего. Тем не менее, все, что создается при выполнении сценария остается в рабочей области.

3.2. Функции

В отличие от этого, функции могут принимать входные аргументы и возвращать результаты. Функции отличаются от других M-файлов по ключевому слову `function` в первой строке кода. Давайте взглянем на простую функцию `acosd`. Код, приведенный ниже, можно увидеть в MATLAB с помощью команды: `type acosd`

```
function y = acosd(x)
%ACOSD  Inverse cosine, result in degrees.
%   ACOSD(X) is the inverse cosine, expressed in degrees,
%   of the elements of X.
%
%   Class support for input X:
%      float: double, single
%
%   See also COSD, ACOS.
```

```
%   Copyright 1984-2004 The MathWorks, Inc.
%   $Revision: 1.1.6.3 $  $Date: 2004/06/25 18:51:38 $

if ~isreal(x)
    error('MATLAB:acosd:ComplexInput', 'Argument should be real.');
end

y = 180/pi*acos(x);
```

Несколько вещей, которые необходимо отметить об этом коде, являются следующие:

- ✓ используется ключевое слово `function`;
- ✓ имя функции является `acosd`. Таким образом, имя М-файла будет `acosd.m`;
- ✓ возвращающее значение, `"y"`, указывается в функции как `y = acosd(x)`;
- ✓ любые аргументы функции указываются в скобках после имени функции. В функции `acosd`, `"x"` является только входным аргументов;
- ✓ все, что следует после `%`, игнорируется как комментарий;
- ✓ первый блок комментариев до первой пустой строки или исполняемой строки отображается, когда вы вводите

  `help acosd`

  (все функции подобны этой).

3.3. Входные и выходные параметры функции

MATLAB функции являются очень гибкими по сравнению со многими другими языками. Они могут иметь несколько и различное количество входных аргументов, что довольно нормально. Они также могут иметь **несколько и различное число выходных результатов**, что может быть новым для вас. Чтобы помочь вам справиться с этой сложностью, MATLAB имеет ряд специальных функций. В следующей таблице приведены следующие функции:

Функция	Описание
`nargin`	Возвращает число входных аргументов в функцию
`nargout`	Возвращает число выходных аргументов из функции

`nargchk(low, high, number)`	Возвращает `"Не достаточно входных параметров"` если `number` меньше, чем `low` и возвращает `"Слишком много входных параметров"` если `number` больше, чем `high`. Если значение `number` находится между (или равно) `low` и `high`, то возвращается пустая матрица. Часто, `nargchk` используется в сочетании `error`. Функция `error` останавливает выполнение и выводит сообщение, если `number` является либо слишком высоким, либо слишком низким. Пример использования `nargchk` внутри функции:
	`error(nargchk(2, 3, nargin));`
	`%This checks if the input arguments (nargin)`
	`%are between 2 and 3 (inclusive).`
	`%If they aren't, then execution stops and`
	`%an error message (with the string returned`
	`%from nargchk) is printed`

`varargin`	принимает переменное число аргументов
`varargout`	возвращает переменное количество аргументов

3.3.1. varargin

Чтобы написать функцию с переменным числом аргументов, вы можете написать:

```
function m=testFunction(varargin)
```

Это означает, что вы могли бы иметь от нуля до того количества аргументов, сколько вы хотите для функции `testFunction`. Следующий код предназначен для проверки этого.

```
function m=testFunction(varargin)
%TESTFUNCTION tests variable number of arguments
%   The following are valid calls to testFunction
%   a=testFunction
%   b=testFunction(val)
%   c=testFunction(val1,val2)
%   d=testFunction(val1,val2,...moreval)


number=nargin;

switch number
    case 0
        m=0;
    case 1
        m=1;
    case 2
        m=2;
    otherwise
        m='more than two';
end
```

Обратите внимание, что нет оператора возврата, вместо этого, `m` (переменная возврата) присваивается значение. Например, если у вас нет аргументы, то `m` будет установлен в 0, если у вас есть один аргумент, то `m` будет установлен в 1, если у вас есть три или более аргументов, то `m` будет присвоено сообщение: `'больше чем два'`.

Если вы хотите получить доступ к любому из аргументу, вы можете использовать обозначение `varargin{#}`. Например, `varargin{1}` будет возвращать первый аргумент. Следующая функция создает `m` в виде массива, который содержит все входные аргументы.

```
function m=testFunction2(varargin)
%TESTFUNCTION2 displays all of arguments sent to the function
%   The following are valid calls to testFunction2
%   a=testFunction
%   b=testFunction(val)
%   c=testFunction(val1,val2)
%   d=testFunction(val1,val2,moreval)


number=nargin;
% m=[];  %this goes with the commented line below


if number>0
    for i=1:number
       m(i)=varargin{i};  %add one more argument to m
       % m=[m varargin{i}];  %this way works as well
    end
```

```
else
    m='no arguments';
end
```

Эта функция использует `if/else` и `for` операторы. Обратите внимание, что `end` используется для завершения этих конструкций. Оператор `for` позволяет использовать оператор двоеточие; в каждом цикле, будет иметься увеличенное значение от 1 до числа аргументов.

Вы можете использовать `varargin` с другими аргументами, но `varargin` должен быть указан последним. Например, если вам требуется один аргумент, а затем с переменным числом аргументов, можно было бы написать так:

```
function m=testFunction3(x, varargin)
```

В этом случае вам потребуется, по крайней мере, один аргумент, и он будет храниться в `x`.

3.3.2. `varargout`

Идея `varargout` недалека от `varargin`. Тем не менее, сначала мы опишем то, что вам может быть не знакомо из других языков – MATLAB может возвращать несколько аргументов. Для того чтобы иметь функцию с несколькими возвращающими значениями, вы можете написать:

```
function [m, n] = testFunction4(x)
```

Это указывает на то, что имеются два возвращающих значения функции, которые будут сохранены в `m` и `n` соответственно.

Вы можете также задать переменное число аргументов, которое должно быть возвращено. Следующий код возвращает переменное количество аргументов в зависимости от количества аргументов, указанных при вызове функции:

```
function [varargout] = testFunction5(x)
%TESTFUNCTION5 tests how variable arguments work
%  testFunction5(x) does not return anything
%  m=testFunction5(x) returns one argument (1+x)
%  [m,n]=testFunction5(x) returns two arguments (1+x and 2+x)
%  [m,n,o]=testFunction5(x) returns three arguments (1+x, 2+x, and 3+x)
%  so on.

number=nargout;

if number>0
    for i=1:number
        varargout{i}=i+x;
    end
end
```

Например, если сделаете следующий вызов:

```
[m,n,o]=testFunction5(1)
```

будет выведено следующее:

```
m =
     2
n =
     3
o =
     4
```

5. Упражнения

Часть 1: Напишите функцию

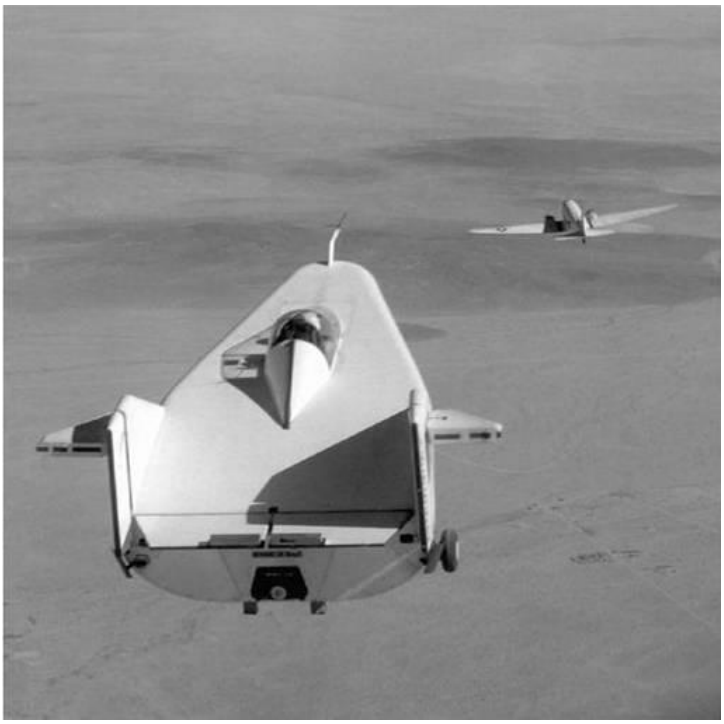
Создайте MATLAB функцию под названием `returnSubImage`. На основании заданного изображения и стартовых позиций *x* и *y*, возвратите квадратный фрагмент изображения. Если размер фрагмента изображения не указан, то по умолчанию размер 100 × 100. Эта функция будет принимать в указанном порядке три обязательных аргумента и необязательный четвертый аргумент:

- ✓ `imageMatrix` (содержит данные об изображении уже считанные из файла);
- ✓ `startingRow` (*y*-значение для начала фрагмента изображения – представляет собой верхний левый угол);
- ✓ `startingCol` (*x*-значение для начала фрагмента изображения – представляет собой верхний левый угол);
- ✓ *`[size of square]`* (будет добавлен к `startingRow` и `startingCol` для получения нижнего правого значения).

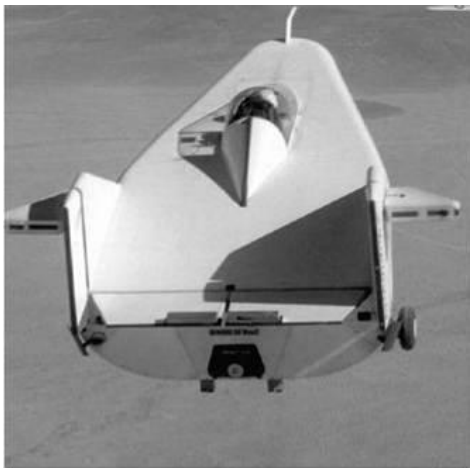
Включите проверку ошибок в вашей функции. Если квадрат за границами исходного изображения, вызовите сообщение об ошибке. Вы можете использовать функцию размера для нахождения размера изображения, с которым вы имеете дело.

Когда вы закончите писать свой код, попробуйте выполнить следующие вызовы функции `returnSubImage`; результаты должны соответствовать тем, что приведены ниже:

1.
```
% чтение изображения для начала действий
lifting = imread('liftingbody.jpg');
imshow(lifting);
```



2.
```
% получение фрагмента изображения ближнего самолета
lifting_sub1 = returnSubImage(lifting, 159, 33, 330);
figure, imshow(lifting_sub1);
```



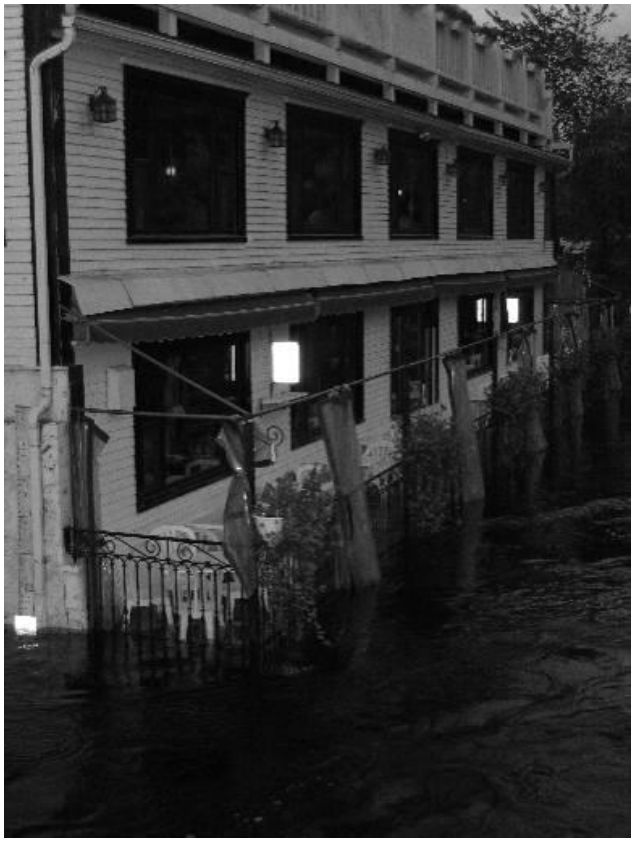
3.
```
% получение фрагмента изображения второго самолета,
% используя квадрат размером 100×100
lifting_sub2 = returnSubImage(lifting, 110, 360);
figure, imshow(lifting_sub2);
```



4.
```
% тестирование на наличие ошибок
lifting_sub=returnSubImage(lifting,159,33,500);
```

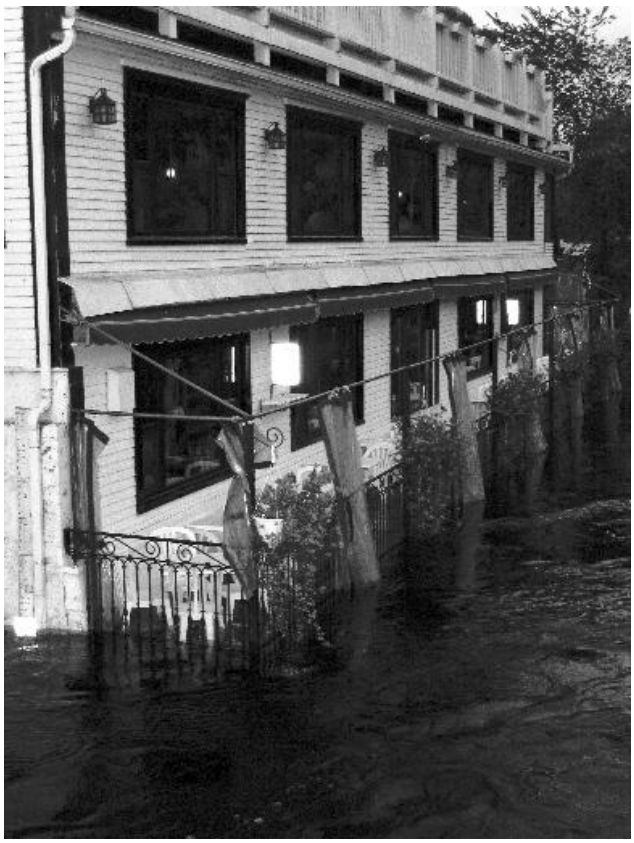
Часть 2: Гистограммы

1. Выберите файл "`flood.jpg`" и сохраните его в текущей директории MATLAB.



2. Загрузите данные об изображении.
3. Отобразите гистограмму изображения.

4. Используя гистограммную эквализацию, создайте новое изображение с большей контрастностью.
5. Отобразите гистограмму изображения, созданного на шаге 4.
6. Примените гистограммную подгонку (с использованием M-файлов, приведенных выше) для получения изображения, которое выглядит примерно следующим образом:



7. Отобразите гистограмму изображения, созданного на шаге 4.

Совет

✓ `manualhist` может быть полезной, но она не является необходимостью. Вы только должны использовать `twomodegauss` функцию для создания соответствующих гистограмм. Она принимает аргументы в том же порядке, как `manualhist`.

Конечный результат

✓ Часть 1
- M-файл под названием `returnSubImage.m`, содержащий реализацию вашей функции.

✓ Часть 2
- M-файл под названием `histograms.m` со скриптом, который отображает следующее (как описано выше):
  - исходное изображение;
  - гистограмму исходного изображения;
  - изображение, полученное на основе гистограммной эквализации;
  - гистограмму изображения, полученного на основе гистограммной эквализации;
  - изображение, полученное на основе гистограммной подгонки;
  - гистограмму изображения, полученного на основе гистограммной подгонки.